

## 2. DNN 심장병 판별하기

1) 실습문제 1/7

---

**임의의 수 생성 알고리즘의 씨앗을 제거해 보세요.**

**어떤 결과를 얻나요?**

1) 실습문제 1/7

---

(1) 문제 분석

- 기계학습에서 가중치의 초기값은 임의의 숫자로 출발
- 가중치의 출발점이 어디인지에 따라, 도착점도 달라짐
- 도착점이 달라지면, 최종 정확도도 달라짐
- 임의의 수 생성 알고리즘의 씨앗이 이를 예방함
  - 매번 실행 때 같은 임의의 수를 생성

(2) 예상 답

- 정확도: 매번 실행시 바뀜
- 학습 시간: 미미한 변화

## 1) 실습문제 1/7

```
DNN 학습 최적화
2 # 2028년 8월 22일
3 # 임성규 교수, 미국 조지아 공대
4 # lissk@ece.gatech.edu
5 # 한국기술교육대학교 온라인 평생교육원
6
7
8 # 라이브러리 패키지 수입
9 import pandas as pd
10 import matplotlib.pyplot as plt
11 import seaborn as sns
12 from time import time
13
14 import torch
15 from torch import nn
16 from sklearn.model_selection import train_test_split
17 from sklearn.preprocessing import StandardScaler
18 from sklearn.metrics import f1_score
19
20
21 # 하이퍼 파라미터
22 INPUT_DIM = 13
23 HY_HIDDEN = 1000
24 HY_EPOCH = 1000
25
26
27 실습문제 1
```

### ◆ 실습문제 1

- 실습에 앞서 화면에 뜨는 상자그림 멈춤
- 임의의 수 생성 알고리즘의 씨앗 확인
- 매번 실행 때 같은 임의의 수를 생성 확인
- 임의의 수 생성 알고리즘의 씨앗을 제거
- 매번 실행 때 다른 임의의 수를 생성함
- 매번 실행 때마다 결과가 달라짐

### (1) 정답

- 정확도 (제거 안함): 0.899, 0.899, 0.899, 0.899
- 정확도 (제거 함): 0.833, 0.891, 0.820, 0.800
- 학습 시간: 6초대

### (2) 추가 문제

- 최적화 함수를 Adam으로 바꾸어 보세요.

## 2) 실습문제 2/7

---

**SGD 대신 RMSprop 최적화 함수를 사용해 보세요.**

**어떤 결과를 얻나요?**

## 2) 실습문제 2/7

---

### (1) 문제 분석

- SGD 최적화 함수는 대부분의 상용 최적화 알고리즘의 원조
- RMSprop은 최적화 도중, 학습율을 상황에 맞게 조절하는 기술을 SGD에 추가
- RMSprop이 SGD보다 더 진화된 알고리즘

### (2) 예상 답

- 정확도: 증가 예상
- 학습 시간: 증가 예상

## 2) 실습문제 2/7

**DNN 학습 최적화**

```

21 # 하이퍼 파라미터
22 INPUT_DIM = 13
23 NY_HIDDEN = 1888
24 NY_EPOCH = 1888
25
26 # 추가 옵션
27 pd.set_option('display.max_columns', None)
28 torch.manual_seed(111)
29 numpy.random.seed(111)
30
31 # 데이터 로드 및 분할
32 # 결과는 pandas의 데이터 프레임 형식
33 raw = pd.read_csv('heart.csv')
34
35 # 데이터의 일부 출력

```

에POCH: 898, 손실: 0.887  
에POCH: 900, 손실: 0.887  
에POCH: 910, 손실: 0.887  
에POCH: 920, 손실: 0.887  
에POCH: 930, 손실: 0.887  
에POCH: 940, 손실: 0.887  
에POCH: 950, 손실: 0.887  
에POCH: 960, 손실: 0.887  
에POCH: 970, 손실: 0.887  
에POCH: 980, 손실: 0.887  
에POCH: 990, 손실: 0.887  
최종 학습 시간: 4.9초

[ True False True True True True True True True  
True False True False True True False True True False  
False False True False False True False True False True  
True False False True True False True True True True  
True True False True False False False True False True  
True False False True False False True False True True  
True False False True True True True True False True  
False False True False True True False]

최종 정확도 (F1 점수): 0.885

실습문제 2

### ◆ 실습문제 2

- 이전 실습문제의 내용을 원상태로 복구
- 최적화 함수 : SGD → RMSprop
- 정확도 : 0.899 → 0.885
- 총 학습시간 : 6.5초 → 19.7초

### (1) 정답

- 정확도: 0.899에서 0.885로 감소
- 학습 시간: 6.5초에서 19.7초로 증가

### (2) 추가 문제

- SGD 대신 Adam 알고리즘을 사용해 보세요.

1) 실습문제 3/7

---

**5,000개의 뉴런을 갖는 새 은닉층을 오른쪽에 추가해  
보세요.**

**어떤 결과를 얻나요?**

1) 실습문제 3/7

---

(1) 문제 분석

- 현재 DNN은 뉴런 1,000개짜리 두개의 은닉층을 가짐
- 여기에 5,000개짜리 하나를 추가함
- 심층신경망에서 은닉층의 숫자가 많을 수록 손실 값은 감소함
- 보정해야 할 가중치의 수도 늘어나 학습시간이 증가함

(2) 예상 답

- 가중치 수: 증가
- 정확도: 증가
- 학습 시간: 증가

## 1) 실습문제 3/7

### DNN 구조 최적화

```

147 nn.Linear(9888, 1),
148 nn.Sigmoid()
149 }
150 print("\nDNN 모델")
151 print(model)
152
153 # 총 파라미터 수 계산
154 total = sum(p.numel() for p in model.parameters())
155 print('총 파라미터 수: {}'.format(total))
156
157 ##### 모델 실행 학습 #####
158
159 # 최적화 함수와 손실 함수 지정
160 optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
161 criterion = nn.MSELoss()
162
163 # 학습을 데이터로 한화
164 # pandas 데이터프레임에서 y_train 변수로
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

```

min -2.286644e+08 -7.379311e-01 -3.648535e+08
25% -6.433722e-01 -7.379311e-01 -4.983575e-01
50% -6.433722e-01 -7.379311e-01 -4.983575e-01
75% 9.999399e-01 2.522843e-01 1.884738e+08
max 9.999399e-01 3.222611e+08 1.884738e+08

```

#### DNN 요약

```

Sequential(
  (0): Linear(in_features=13, out_features=1888, bias=True)
  (1): Tanh()
  (2): Linear(in_features=1888, out_features=1888, bias=True)
  (3): Tanh()
  (4): Linear(in_features=1888, out_features=5888, bias=True)
  (5): Tanh()
  (6): Linear(in_features=5888, out_features=1, bias=True)
  (7): Sigmoid()
)
총 파라미터 수: 6,025,881

```

#### DNN 학습 시작

```

에포크: 0, 손실: 8.247
에포크: 10, 손실: 8.198
에포크: 20, 손실: 8.163

```

실습문제 3

### ◆ 실습문제 3

- 이전 실습문제의 내용을 원상태로 복구
- 5,000개의 뉴런을 갖는 새 은닉층의 추가
- 가중치 수: 1,016K → 6,025K
- 총 학습 시간: 6.5초 → 33.4초
- 정확도: 0.899 → 0.901

### (1) 정답

- 가중치 수: 1,016K에서 6,025K로 증가
- 정확도: 0.899에서 0.901로 증가
- 학습 시간: 6.5초에서 33.4초로 증가

### (2) 추가 문제

- 원래 DNN에서 두 번째 은닉층을 삭제해 보세요.

### Tanh 활성화 함수 대신

ReLU 활성화 함수를 사용해 보세요.

어떤 결과를 얻나요?

#### (1) 문제 분석

- Tanh 활성화는 뉴런 출력값을  $[-1, 1]$ 로 조절
- 심장병 데이터는 Z-점수 정규화 처리함  
-평균은 0, 표준편차는 1
- 두 가지 (입력 데이터와 뉴런 출력값) 범위가 비슷
- ReLU는 뉴런 출력값을  $[0, \infty)$ 로 조절

#### (2) 예상 답

- 가중치 수: 변함 없음
- 정확도: 감소
- 학습 시간: 영향 미미

## 2) 실습문제 4/7

### DNN 구조 최적화

```

192 plt.show()
193
194 ##### 기존 신경망 구조 #####
195
196 # 자이보치 2018을 Sequential 모델로 구현
197 model = nn.Sequential(
198     nn.Linear(INPUT_DIM, HY_HIDDEN),
199     nn.Tanh(),
200     nn.Linear(HY_HIDDEN, HY_HIDDEN),
201     nn.Tanh(),
202     nn.Linear(HY_HIDDEN, 5000),
203     nn.Tanh(),
204     nn.Linear(5000, 1),
205     nn.Sigmoid()
206 )
207 print('\nDNN 요약')
208 print(model)
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

```

예측값: 898, 손실: 0.100
예측값: 988, 손실: 0.100
예측값: 918, 손실: 0.100
예측값: 928, 손실: 0.100
예측값: 938, 손실: 0.100
예측값: 948, 손실: 0.100
예측값: 958, 손실: 0.100
예측값: 968, 손실: 0.107
예측값: 978, 손실: 0.107
예측값: 988, 손실: 0.107
예측값: 998, 손실: 0.107
최종 학습 시간: 33.4초
[ True True False True True True False True False True
 True True False False True False True False True False
 True False True False True False True False True False
 False True False False False False False True True True
 True True False False True False True True True True
 False True False True False False True False True True
 True True True True False False False False True True
 True True True False True True True]
최종 정확도 (F1 점수): 0.981

```

실습문제 4

- ◆ 실습문제 4
  - 이전 실습문제의 내용을 원상태로 복구
  - 활성화 함수 : Tanh → ReLU
  - 총 학습 시간 : 6.5초 → 8.2초
  - 정확도 : 0.899 → 0.937

### (1) 정답

- 가중치 수: 변함 없음
- 정확도: 0.899에서 0.937로 증가
- 학습 시간: 6.5초에서 8.2초로 증가

### (2) 추가 문제

- Tanh 활성화를 Sigmoid로 바꾸어 보세요.



**Z-점수 정규화를 건너 뛰어 보세요.**

**어떤 결과를 얻나요?**

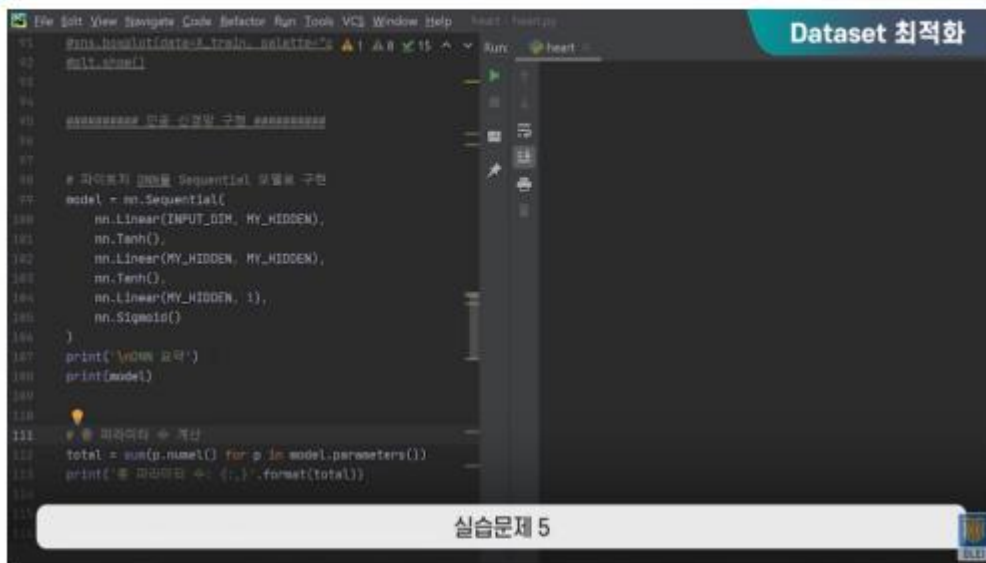
(1) 문제 분석

- Z-점수 정규화는 기계학습에 사용된 14개의 모든 요소들을 Z-점수화함
- 이를 통해 14개 요소간의 영향력이 비슷해 짐
- 이를 건너뛰었을 때, 숫자가 큰 요소들의 중요도가 커짐
- 이는 바람직하지 않음

(2) 예상 답

- 데이터 수: 변함 없음
- 정확도: 감소
- 학습 시간: 영향 미미

## 1) 실습문제 5/7



```
File Edit View Help Code Refactor Run Tools VCS Window Help heart.py Dataset 최적화
171 from keras.datasets import mnist
172 from keras.models import Sequential
173 from keras.layers import Dense, Activation
174
175 # 데이터 로드 및 전처리
176 (train_images, train_labels), (test_images, test_labels) = mnist.load_data()
177
178 # 데이터 정규화
179 train_images = train_images.reshape((train_images.shape[0], 784))
180 train_images = train_images.astype('float32') / 255
181 test_images = test_images.reshape((test_images.shape[0], 784))
182 test_images = test_images.astype('float32') / 255
183
184 # 모델 구성
185 model = Sequential()
186 model.add(Dense(10, input_dim=784))
187 model.add(Activation('tanh'))
188 model.add(Dense(10))
189 model.add(Activation('tanh'))
190 model.add(Dense(1))
191 model.add(Activation('sigmoid'))
192
193 # 모델 요약
194 print(model.summary())
195
196 # 파라미터 개수 계산
197 total = sum(p.size() for p in model.parameters())
198 print('총 파라미터 수: {}'.format(total))
```

### ◆ 실습문제 5

- 이전 실습문제의 내용을 원상태로 복구
- Z-점수 정규화를 건너뛰기
- 정규화 여부 확인하기
- 데이터 수 : 변함 없음
- 데이터의 순서가 랜덤화됨
- 총 학습 시간 : 6.5초 → 6.8초
- 정확도 : 0.899 → 0.737

### (1) 정답

- 데이터 수: 변함 없음
- 정확도: 0.899에서 0.737로 감소
- 학습 시간: 6.5초에서 6.8초로 비슷

### (2) 추가 문제

- StandardScaler를 MinMaxScaler로 바꾸어 보세요.

## 2) 실습문제 6/7

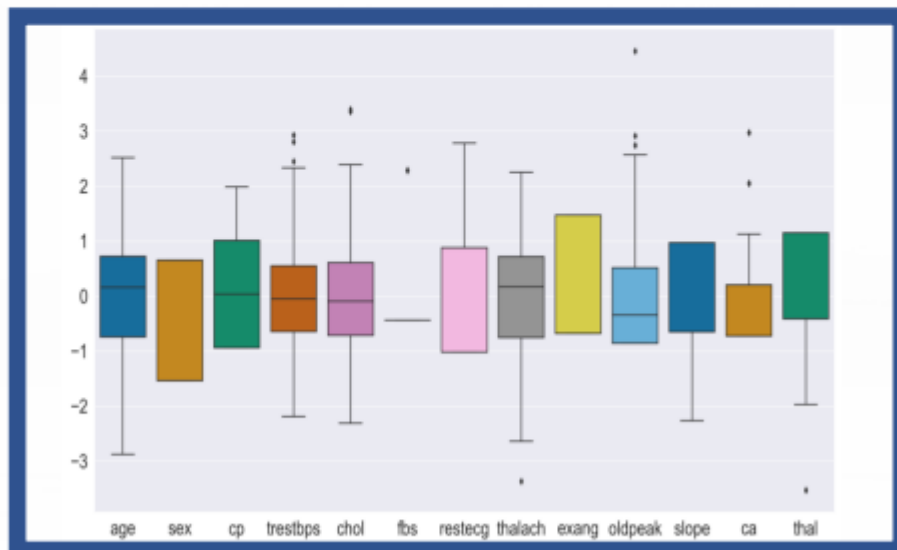
심장병 데이터에서, fbs와 chol 컬럼을 삭제해 보세요.

어떤 결과를 얻나요?

## 2) 실습문제 6/7

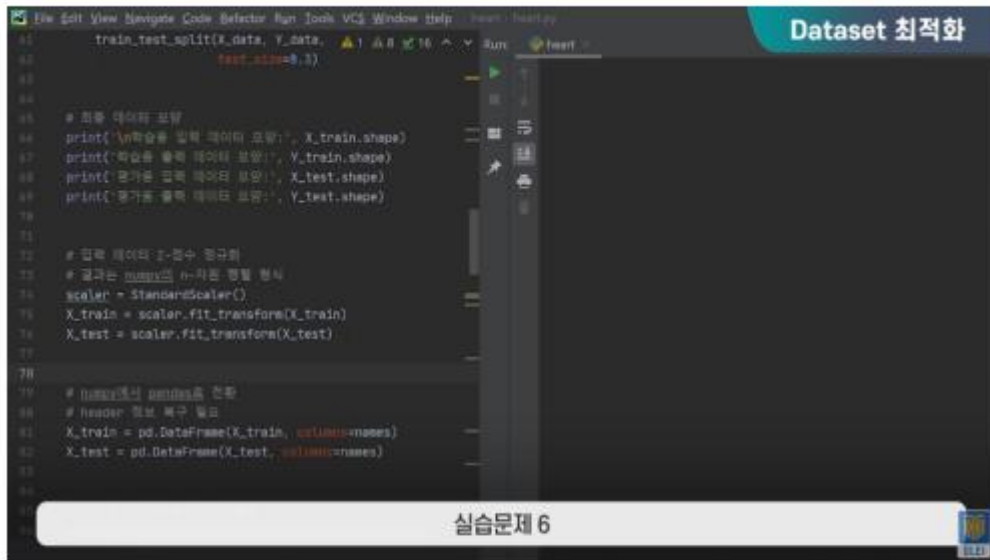
### (1) 문제 분석

- Dataset에서 어떤 요소는 기계학습에 방해
- fbs (공복 혈당)과 chol (좋은 콜레스테롤) 두 요소에서 특이점 다수 발견



- 방해 데이터가 없어지면 정확도와 학습 시간에 긍정적

## 2) 실습문제 6/7



```
1 train_test_split(X_data, Y_data, test_size=0.3)
2
3 # 최종 데이터 모양
4 print('\n학습용 입력 데이터 모양:', X_train.shape)
5 print('\n학습용 출력 데이터 모양:', Y_train.shape)
6 print('\n평가용 입력 데이터 모양:', X_test.shape)
7 print('\n평가용 출력 데이터 모양:', Y_test.shape)
8
9 # 입력 데이터 1-컬럼 정규화
10 # 결과는 numpy의 n-차원 배열 형식
11 scaler = StandardScaler()
12 X_train = scaler.fit_transform(X_train)
13 X_test = scaler.fit_transform(X_test)
14
15 # numpy에서 pandas로 변환
16 # header 정보 복구 필요
17 X_train = pd.DataFrame(X_train, columns=names)
18 X_test = pd.DataFrame(X_test, columns=names)
```

### ◆ 실습문제 6

- 이전 실습문제의 내용을 원상태로 복구
- 심장병 데이터에서 fbs와 chol 삭제
- 인풋 디멘션 : 13 → 11
- 데이터 수 : (212, 13) → (212, 11)
- 총 학습 시간 : 6.5초 → 7.0초
- 정확도 : 0.899 → 0.919

### (1) 정답

- 데이터 수: (212, 13)에서 (212, 11)로 감소
- 정확도: 0.899에서 0.919로 증가
- 학습 시간: 6.5초에서 7.0초로 비슷

### (2) 추가 문제

- trestbps와 ca 요소들을 추가 삭제해 보세요.

**303개의 데이터 중 처음 100개만 사용해 보세요.**

**어떤 결과를 얻나요?**

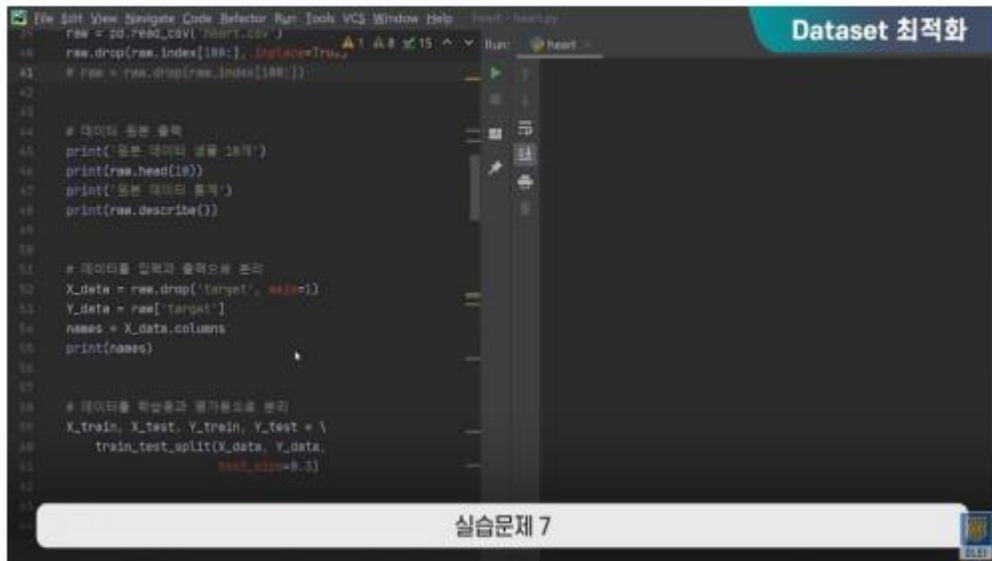
**(1) 문제 분석**

- 기계학습에 있어 데이터의 양과 질이 기계학습의 성패를 크게 좌우함
- 데이터에서 처음 100만 사용하면 그만큼 정확도에 나쁜 영향을 미침
- 다만 학습 시간은 그에 비례하여 감소

**(2) 예상 답**

- 데이터 수: 감소
- 정확도: 감소
- 학습 시간: 감소

## 2) 실습문제 7/7



```
17 raw = pd.read_csv('heart.csv')
18 raw.drop(raw.index[188], inplace=True)
19 # raw = raw.drop(raw.index[188:])
20
21
22 # 데이터의 기본 출력
23 print('기본 데이터 샘플 10개')
24 print(raw.head(10))
25 print('기본 데이터 통계')
26 print(raw.describe())
27
28
29 # 데이터를 입력과 출력으로 분리
30 X_data = raw.drop('target', axis=1)
31 Y_data = raw['target']
32 names = X_data.columns
33 print(names)
34
35
36 # 데이터를 학습용과 평가용으로 분리
37 X_train, X_test, Y_train, Y_test = \
38     train_test_split(X_data, Y_data,
39                     test_size=0.5)
```

### ◆ 실습문제 7

- 이전 실습문제의 내용을 원상태로 복구
- 303개의 데이터 중 처음 100개만 사용
- 방법 1 : 드랍함수 사용
- 방법 2 : 로 드랍
- 데이터 수: 303 → 100
- 총 학습 시간: 6.5초 → 4.3초
- 정확도 : 0.899 → 1.0

### (1) 정답

- 데이터 수: 303개에서 100으로 감소
- 정확도: 0.899에서 1.0으로 증가
- 학습 시간: 6.5초에서 4.3초로 감소

### (2) 추가 문제

- 새로운 학습용 데이터를 직접 만들어서 추가해 보세요.